1. Project Scope

The Class Planner Web Application is designed to help students manage their workload by organizing classes and tracking assignments, and to remind them of completing their assignments.

In Scope

- Users can sign and log in to their credentials
- Users can create and delete classes
- Users can add assignments for the specific classes and view them
- Users can also see assignments due within the next 7 days and upcoming assignments
- Users can mark assignments as completed and view them
- The system will provide basic reminders for upcoming assignments
- Data will be stored and persisted using a database

Out of Scope

- No mobile version
- Real-time notifications
- Advance AI scheduling

2. Constraints

Built using:

- Frontend: HTML, CSS, JavaScript
- Backend: Python (Flask)
- Database: MySQL
- Runs in a web browser (no native mobile support)
- Limited experience level (beginner-friendly tools only)

Resource Constraints:

- Developed by a single developer
- No paid tools or services

Performance Constraints

- Should handle basic usage for at most a small group of users at the same time

3. Assumptions

- Users have access to a modern web browser
- Users will be able to input their classes and assignments manually
- Users will be able to remove classes and mark assignments as done
- An internet connection is available when using the app
- Users only need basic reminder functionality (not complex scheduling)

4. Functional Requirements

- Authentication
    - The system shall allow users to register an account and allow them to log in/out
    - The system shall securely store user credentials for users to log in

- Class Management
    - The system shall allow users to create, edit, and delete classes

    - The system shall display all classes for the logged-in user

- Assignment Management
    - The system shall allow users to create and edit assignments linked to a class
    - The system shall let users edit the status of the assignment to say if it's completed
    - Each assignment shall include:
        - Title
        - Due date and time
        - Type (Homework, Exam, Project, Quiz)

- Assignment Tracking
    - The system shall display assignments due within the next 7 days
    - The system shall display all upcoming assignments
    - The system shall let users edit the status of the assignment to say if it's completed/incompleted

- Reminder Generation
    - The system shall provide a “Generate Reminders” option in the menu
    - The system shall display reminders based on:
        - Due dates and times
        - Assignment type (Homework, Quizzes, Test, Projects)
    - The system shall prioritize reminders (e.g., Exams before Homework)
    - The system shall rank reminders using:
        - Due date proximity (closer = higher priority)
        - Assignment type weight (Exam > Project > Homework)
        - User-defined priority level

5. Non-Functional Requirements

- Performance
    - The system should load within 2–3 seconds
    - Reminder generation should complete within 1 second
    - The system shall let multiple people use the system simultaneously
- Security
    - User passwords must be encrypted
    - Only authenticated users can access their data and credentials
- Usability
    - The interface should be simple and intuitive
    - Users should be able to perform core actions within 3 clicks
- Compatibility
    - The system should work on modern browsers (Chrome, Edge, Safari)

6. Personas

- Persona 1: Busy College Student
    - Name: Alex
    - Age: 20
    - Major: Computer Science
    - Problem:
        - Has multiple assignments across different classes
        - Often forgets deadlines
    - Goal:
        - Wants a simple way to track assignments and deadlines
- Persona 2: Organized Planner
    - Name: Sarah

    - Age: 22
    - Major: Business
    - Problem:
        - Keeps track manually but wants something more efficient
    - Goal:
        - Wants categorized assignments and reminder summaries

7. User Stories

- Classes:
    - As a user, I want to be able to add my classes to organize and separate my coursework for different subjects
    - As a user, I want to be able to delete a class that I am not taking anymore to have more accurate reminders
- Assignment:
    - As a user, I want to be able to add assignments from my classes to track my work
    - As a user, I want to be able to give the status of my assignments as completed, so I can keep track of what is finished
- Tracking
    - As a user, I want to be able to track my progress on how many assignments are due to complete them on time
    - As a user, I want to be able to see how many assignments are due on a certain date so I can plan what day to do those assignments instead of waiting until the last minute
- Reminders
    - As a user, I want to be able to generate reminders to be informed of what assignments I should start

- As a user, I want important assignments (like exams or assignments that are overdue) to appear first so that I can prioritize

8. System Architecture Decisions

- Frontend
    - HTML (structure)
    - CSS (styling)
    - JavaScript (logic, API calls)
- Backend
    - Python + Flask
- Database
    - MySQL

9. Database Design:

User table:

Field	Type	Notes
ID	INT (PK)	Unique user IDs
Username	Varchar	Unique
Email	Varchar	Unique
Password	Varchar	Hashed
Major	Varchar	Major of the users
Concentration	Varchar	Concentration of the Major

Class Table:

Field	Type	Notes
ID	INT (PK)	Class IDs
User_ID	INT (FK)	Links to Users
Name	Varchar	Class Name
Subject	Varchar	Subject of the class

Assignment Table:

Field	Type	Notes
ID	INT (PK)	Assignment ID
Class_ID	INT (FK)	Links to classes
title	Varchar	Name of the assignment
due_date	DATETIME	Due date of assignment
type	Varchar	Homework, quiz, test, project
priority	Int	(1=High, 2=Medium, 3=Low)
completed	Boolean	True/False

10. Relationships:

One user → many classes

One class → many assignments

11. Backend API Design

3. Backend API Design (Flask)

- Auth Routes
    - POST /register
    - POST /login
    - POST /logout
- Class Routes
    - GET /classes
    - POST /classes
    - PUT /classes/<id>
    - DELETE /classes/<id>
- Assignment Routes
    - GET /assignments
    - POST /assignments
    - PUT /assignments/<id>
- Reminder Route (Your Feature)
    - GET /reminders/generate

12. UI/UX Design Structure/Rationale

Main Screens

- Login / Register Page
    - Username/email input
    - Password input
    - Login + Register buttons
- Dashboard (Main Page)
    - The Dashboard serves as the **central hub** of the application, allowing users to quickly navigate core features and view time-sensitive information.
- Sections

- Create Classes and Assignments
    - A button that redirects users to a dedicated page for:
    - Creating new classes
    - Adding assignments to those classes
- Rationale:
    - Separating creation into its own page reduces clutter on the dashboard and keeps the main interface focused on overview and navigation.

- View Classes and Assignments
    - A button that redirects users to a page where they can:
        - View all classes
        - View assignments within each class
        - Edit or delete existing classes and mark completed for assignments
    - Rationale:
        - Provides a centralized location for managing existing data without overwhelming the dashboard.

- Upcoming Assignments (Next 7 Days)
    - Displays a list of assignments due within the next 7 days
    - Each item includes:
        - Assignment title
        - Associated class
        - Due date and time
        - Priority indicator

    - Rationale:
        - Helps users quickly identify urgent tasks and stay on track with upcoming deadlines.

- Generate Reminders
    - A button that, when clicked:
        - Generates a prioritized list of assignments
    - Sorts based on:
        - Due date
        - Assignment type
        - Priority level
    - Rationale:
        - Allows users to actively generate reminders when needed, avoiding unnecessary notifications while still supporting planning and prioritization.

13. User Flow

- User can log in
- Look at the dashboard
- Be able to see the upcoming assignments
- Generate new classes and assignments

- Mark assignments as complete
- Generate reminders on when to do assignments
- Start on those assignments

14. Testing Strategy

- Manual testing will be used to validate:
    - User registration and login with credentials
    - Creating classes
    - Adding assignments
    - Marking assignments as complete
    - Generating reminders
- Edge cases:
    - Empty inputs
    - Invalid date/time entries
    - Unauthorized access

AI tools were used to generate test cases and validate expected system behavior.